

Chess Bot v1 : un Transformer joue aux échecs sans recherche

SOMMAIRE

- 01 Résumé
- 02 Abstract
- 03 Sommaire
- 04 1. Introduction
- 05 2. Présentation du sujet et analyse du besoin
- 06 3. Méthodologie et organisation du projet
- 07 4. Réalisation du projet
- 08 5. Tests et résultats
- 09 6. Difficultés rencontrées et solutions apportées
- 10 7. Bilan personnel et compétences acquises
- 11 8. Conclusion et perspectives
- 12 9. Bibliographie et sources
- 13 10. Annexes et GitHub

Projet de substitution au stage

SEIBOU Naadjath & LAKRA Rajaa

ECE (École Centrale d'Électronique), Formation : Bachelor Développeur d'Application

Campus : ECE Paris, Groupe : [à vérifier sur votre espace étudiant]

Année universitaire 2025-2026, dépôt le 30 août 2026, soutenance semaine du 31 août 2026

Ce rapport suit le plan officiel du projet de substitution au stage. Les passages entre crochets [...] sont à compléter. Tous les chiffres déjà présents sont réels et proviennent de nos propres expériences.

Résumé

Ce projet explore une question issue d'un article récent de DeepMind (*Grandmaster-Level Chess Without Search*, 2024) : un réseau de neurones de type Transformer peut-il jouer aux échecs en choisissant directement un coup à partir de la position, sans explorer l'arbre des variantes ?

Nous avons construit une chaîne complète : extraction de parties de la base ouverte Lichess, encodage des positions, entraînement d'un Transformer encodeur de 6,86 millions de paramètres par clonage comportemental, puis évaluation du niveau de jeu en Elo face à des adversaires de référence et à Stockfish bridé.

Notre modèle atteint une exactitude de prédiction (top-1) de 22,7 % sur des positions jamais vues, et un niveau de jeu estimé à environ 230-300 Elo (niveau grand débutant). Il apprend visiblement des principes d'ouverture : dans la position de départ, il propose spontanément des coups classiques (Cf3, Cc3, d4, c4) sans qu'aucune règle ne lui ait été enseignée, mais reste trop faible pour convertir ses positions gagnantes. Au-delà de la performance brute, ce travail met l'accent sur la rigueur de la mesure (intervalles de confiance de Wilson, alternance des couleurs, jeux de validation séparés par partie) et sur la reproductibilité.

Mots-clés : apprentissage automatique, Transformer, échecs, clonage comportemental, évaluation Elo.

Abstract

This project investigates a question raised by a recent DeepMind paper (*Grandmaster-Level Chess Without Search*, 2024) : can a Transformer neural network play chess by choosing a move directly from the board position, without ever searching the game tree ?

We built a complete pipeline : extracting games from the open Lichess database, encoding board positions, training a 6.86-million-parameter Transformer encoder through behavioral cloning, and evaluating the resulting playing strength in Elo against reference bots and a strength-limited Stockfish.

Our model reaches a top-1 prediction accuracy of 22.7% on unseen positions, and an estimated playing strength of roughly 230-300 Elo (beginner level). It visibly learns opening principles : from the starting position, it spontaneously proposes classical moves (Nf3, Nc3, d4, c4) despite never being taught a single rule of chess, but remains too weak to convert winning positions. Beyond raw performance, this work emphasizes measurement rigor (Wilson confidence intervals, alternating colors, game-level train/validation splits) and reproducibility.

Keywords: machine learning, Transformer, chess, behavioral cloning, Elo evaluation.

Sommaire

1. Introduction
2. Présentation du sujet et analyse du besoin
3. Méthodologie et organisation du projet
4. Réalisation du projet
5. Tests et résultats
6. Difficultés rencontrées et solutions apportées
7. Bilan personnel et compétences acquises
8. Conclusion et perspectives
9. Bibliographie et sources
10. Annexes et GitHub

1. Introduction

1.1 Contexte

Les échecs sont un terrain d'expérimentation historique pour l'intelligence artificielle. En 1997, Deep Blue bat Garry Kasparov grâce à une recherche massive dans l'arbre des coups. En 2017, AlphaZero atteint un niveau surhumain en combinant un réseau de neurones et une recherche arborescente de Monte-Carlo (MCTS). Aujourd'hui, le moteur libre Stockfish domine le jeu en associant une recherche alpha-bêta très optimisée à un petit réseau d'évaluation (NNUE).

Ces approches ont un point commun : elles cherchent. Elles explorent des milliers, voire des millions de positions futures avant de décider d'un coup.

En 2024, DeepMind publie *Grandmaster-Level Chess Without Search*. L'idée est radicale : entraîner un Transformer à choisir un coup sans aucune recherche, et montrer qu'il peut atteindre un niveau de grand maître. Le réseau « regarde » la position et répond, comme un joueur humain fort qui joue en blitz à l'intuition.

1.2 Choix du sujet et problématique

Ce sujet nous a été proposé dans la liste des projets de substitution au stage (« Chess Bot v1 »). Nous l'avons choisi parce qu'il combine un véritable défi d'apprentissage automatique (entraîner et évaluer un Transformer) avec un domaine où l'intuition est facile à vérifier : on peut littéralement regarder le bot jouer et juger s'il « comprend » quelque chose.

Notre problématique reprend la question de DeepMind, à notre échelle :

Un Transformer de taille modeste, entraîné avec des moyens limités, peut-il apprendre à jouer aux échecs de façon raisonnable, sans jamais explorer l'arbre des coups ?

1.3 Objectifs et démarche générale

- Concevoir et entraîner un Transformer qui choisit un coup à partir d'une position, sans recherche.
- Mesurer son niveau de façon rigoureuse en Elo, face à des adversaires de référence et à Stockfish bridé.
- Livrer une application permettant de jouer contre le bot et d'observer son raisonnement.
- Documenter la démarche avec la même exigence qu'un travail de laboratoire : chaque choix doit être justifié et chaque résultat mesuré avec son incertitude.

Nous assumons dès le départ une version réduite de l'approche DeepMind : là où l'article original utilise 270 millions de paramètres, 10 millions de parties et du matériel spécialisé (TPU), nous disposons d'un GPU grand public gratuit (Google Colab) et de quelques semaines. Cette contrainte est détaillée en partie 2.

2. Présentation du sujet et analyse du besoin

2.1 Le problème étudié

L'énoncé du projet demande de concevoir un bot d'échecs basé sur l'architecture Transformer, capable d'être évalué face à Stockfish. Le problème technique central est celui de la **représentation** : un Transformer, conçu à l'origine pour le texte, ne « voit » ni un échiquier ni une pièce. Il faut donc :

1. transformer une position d'échecs en une entrée numérique exploitable ;
2. transformer la sortie du réseau (des scores) en un coup légal ;
3. mesurer objectivement si le résultat « joue aux échecs » ou non.

2.2 Utilisateurs et bénéficiaires

Ce projet n'a pas vocation commerciale ; ses bénéficiaires directs sont :

- **le jury et l'équipe pédagogique**, qui évaluent notre capacité à mener un projet de recherche appliquée en autonomie ;

- **nous-mêmes**, en tant qu'exercice de montée en compétence sur les architectures Transformer et l'apprentissage automatique appliqué à un domaine à contraintes strictes (règles du jeu, légalité des coups) ;
- plus largement, **toute personne curieuse** de voir concrètement ce qu'un petit modèle de langage-like peut apprendre d'un jeu de règles, via l'application livrée.

2.3 Contraintes du projet

CONTRAINTE	DÉTAIL	CONSÉQUENCE SUR NOS CHOIX
Matériel	GPU gratuit (Google Colab), sessions limitées à ~30-40 min avant déconnexion	Modèle volontairement petit (6,86 M paramètres), sauvegarde à chaque époque
Temps	Quelques semaines, à deux, en parallèle d'autres projets	Portée réduite par rapport à l'article DeepMind, priorité à la rigueur plutôt qu'à la performance brute
Données	Pas de dataset annoté fourni	Construction d'un pipeline d'extraction depuis la base ouverte Lichess
Évaluation	Aucun Elo officiel disponible pour un bot non déployé	Mesure par matchs simulés contre des adversaires de référence et Stockfish bridé, avec intervalle de confiance
Livrables imposés	Application fonctionnelle, Elo approximatif, code documenté, rapport, documentation	Structure du projet organisée autour de ces 5 livrables dès le départ

2.4 État de l'art

SYSTÈME	ANNÉE	PRINCIPE	RECHERCHE ?
Deep Blue	1997	recherche alpha-bêta massive + évaluation experte	Oui (énorme)
AlphaZero	2017	réseau de neurones + MCTS, appris par auto-apprentissage	Oui (MCTS)
Leela Chess Zero	2018+	réimplémentation ouverte d'AlphaZero	Oui (MCTS)
Stockfish (NNUE)	2020+	alpha-bêta + petit réseau d'évaluation	Oui (alpha-bêta)
DeepMind searchless	2024	Transformer seul, aucune recherche	Non

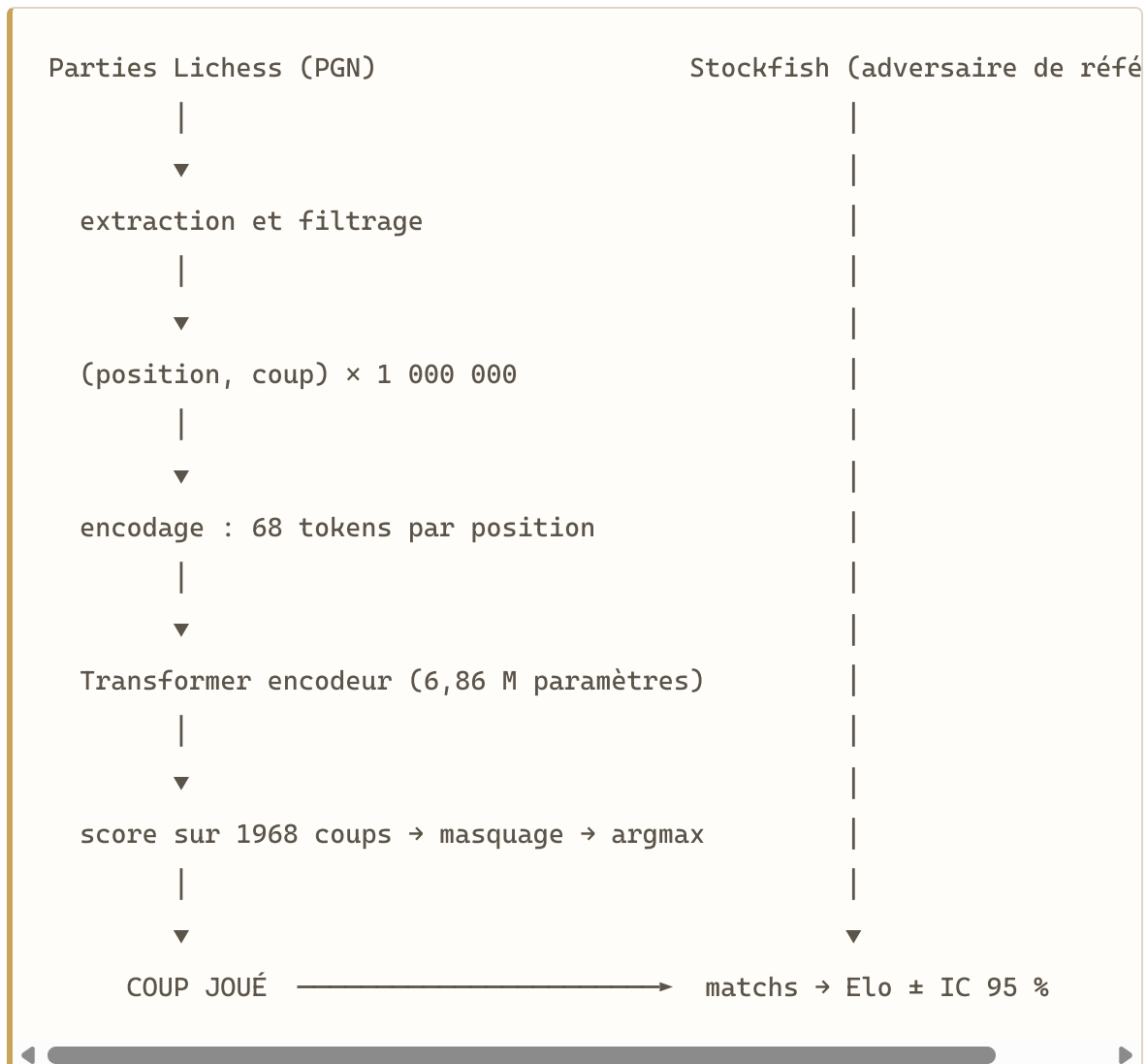
L'architecture **Transformer** (Vaswani et al., 2017) a été conçue pour le traitement du langage. Son mécanisme d'**attention** permet à chaque élément d'une séquence de prendre en compte tous les autres, en une seule étape. Nous exploitons cette propriété en traitant l'échiquier comme une séquence de 64 cases : l'attention relie alors directement deux cases éloignées, ce qui correspond à la portée réelle des pièces (une tour ou un fou agissent à distance, pas seulement sur les cases voisines).

Le **clonage comportemental** (behavioral cloning) consiste à apprendre à imiter les décisions d'un expert à partir d'exemples, sans mécanisme de recherche ni de récompense explicite. Sa limite théorique est le niveau de l'expert imité, d'où l'importance de sélectionner des parties de joueurs forts (voir §4.1). C'est la méthode que nous avons retenue, par opposition à l'apprentissage par renforcement (utilisé par AlphaZero) qui aurait demandé des moyens de calcul hors de portée pour ce projet.

L'article de référence de DeepMind (Ruoss et al., 2024) va plus loin que le clonage comportemental simple : il entraîne le modèle à prédire une *action-value* (la probabilité de gagner après chaque coup), annotée par Stockfish à pleine puissance, plutôt que d'imiter le coup humain. Cette distinction explique une bonne partie de l'écart de performance entre leur modèle et le nôtre ; nous y revenons en perspective (§8).

3. Méthodologie et organisation du projet

3.1 Vue d'ensemble de la chaîne



3.2 Méthode de travail et outils

- **Gestion de version.** Le projet est versionné sur GitHub (`github.com/naadjath/chess-bot-v1`), avec des commits réguliers des deux membres du binôme, ce qui permet de retracer la progression et la répartition réelle du travail.
- **Entraînement.** Google Colab (GPU T4 gratuit), avec sauvegarde automatique des poids sur Google Drive à chaque époque pour survivre aux déconnexions fréquentes de l'environnement gratuit.
- **Tests.** Suite de 32 tests automatisés (`pytest`), couvrant l'encodage, le vocabulaire de coups, le parseur PGN et le modèle, exécutée avant chaque étape importante pour éviter de propager un bug silencieux dans les données ou l'entraînement.
- **Documentation continue.** Chaque module du code contient une explication du *pourquoi* du choix technique, pas seulement du *quoi*, pour que le rapport et le code restent cohérents.

3.3 Organisation du binôme et planning

Le travail n'a pas été séparé selon une frontière stricte (par exemple « une personne sur les données, l'autre sur l'application ») : nous avons avancé en binôme sur l'ensemble de la chaîne, en nous répartissant les tâches au fur et à mesure selon les priorités du moment et les disponibilités de chacune.

ÉTAPE	CONTENU
1. Cadrage	Lecture de l'article DeepMind, compréhension du sujet, choix d'architecture
2. Fondations	Vocabulaire des coups, encodage des positions, tests de réversibilité
3. Données	Parseur PGN, pipeline d'extraction, moteurs de référence (aléatoire, glouton, minimax)
4. Modèle	Implémentation et entraînement du Transformer
5. Évaluation	Intégration de Stockfish, calcul d'Elo, application de jeu
6. Finalisation	Campagne d'évaluation complète, rédaction du rapport

Ces étapes se sont enchaînées sur la durée du projet, sans dates figées à l'avance : chaque étape a démarré une fois la précédente suffisamment avancée pour s'appuyer dessus.

3.4 Justification des principaux choix méthodologiques

- **Clonage comportemental plutôt qu'apprentissage par renforcement :** l'auto-apprentissage façon AlphaZero demande des dizaines de milliers de parties générées par le modèle lui-même, donc un budget de calcul très supérieur à un GPU gratuit. Le clonage comportemental permet d'apprendre directement à partir de parties humaines déjà jouées.
- **Un Transformer encodeur plutôt qu'un réseau convolutif (CNN) :** un CNN raisonne d'abord localement (cases voisines), alors qu'aux échecs une pièce comme la tour ou le fou agit à longue distance. L'attention du Transformer relie n'importe quelle case à n'importe quelle autre dès la première couche.

- **Filtrage strict des données ($Elo \geq 2000$, cadence lente)** : apprendre d'un joueur faible ou pressé par le temps aurait borné la qualité du modèle plus bas encore ; voir §4.1 pour le détail des filtres.
 - **Une échelle volontairement réduite** : plutôt que de viser une performance élevée hors de portée de nos moyens, nous avons choisi de prioriser la rigueur de la démarche et de la mesure, quitte à obtenir un niveau de jeu modeste (voir §5).
-
-

4. Réalisation du projet

4.1 Les données

Source. Base ouverte Lichess (`database.lichess.org`), archive mensuelle de janvier 2024. Ces fichiers contiennent l'intégralité des parties jouées sur le site, au format PGN compressé (`.pgn.zst`).

Lecture en flux. Une archive décompressée pèse plusieurs dizaines de gigaoctets. Plutôt que de tout télécharger, nous lisons le fichier en flux et nous arrêtons dès que le nombre voulu de positions est atteint. Un lecteur optimisé qui rejette une partie sur ses en-têtes avant d'analyser ses coups nous a permis d'atteindre un débit de $\sim 7\,500$ parties/seconde, ramenant la préparation d'un million de positions à moins de 5 minutes (contre plusieurs heures pour une implémentation naïve, voir §6).

Filtres appliqués :

FILTRE	VALEUR	JUSTIFICATION
Elo minimum des deux joueurs	2000	apprendre d'un joueur fort
Cadence minimale	180 s	exclure le bullet, où l'on joue à l'instinct
Partie terminée	oui	une partie abandonnée n'apprend rien
Ouverture ignorée	8 premiers demi-coups	l'ouverture relève de la mémorisation
Plafond par partie	200 demi-coups	équilibrer le poids des parties longues

Statistiques réelles de notre extraction (janvier 2024) :

- Parties lues : 186 057
- Parties retenues : 14 456 (taux de rétention 7,8 %)
- Positions produites : 1 000 012
- Découpage : 980 440 pour l'entraînement, 19 572 pour la validation

Séparation entraînement/validation par partie. Deux positions d'une même partie se ressemblent beaucoup. Les répartir de part et d'autre serait une fuite de données : le modèle serait évalué sur des situations quasiment déjà vues, ce qui gonflerait artificiellement ses scores. Nous répartissons donc des parties entières, jamais des positions isolées.

4.2 L'encodage d'une position

Une position est transformée en 68 nombres entiers :

- **tokens 0 à 63** : le contenu des 64 cases (0 = vide, 1 à 6 = pièces du joueur au trait, 7 à 12 = pièces adverses) ;
- **token 64** : le trait ;
- **tokens 65-66** : les droits de roque ;
- **token 67** : la case de prise en passant.

Normalisation de couleur. Le modèle apprend toujours du point de vue du joueur au trait, comme s'il jouait les blancs. Quand c'est aux noirs de jouer, l'échiquier est retourné (symétrie verticale et inversion des couleurs) et le coup prédit est retourné en retour. Le modèle n'a ainsi qu'un seul point de vue à apprendre, ce qui double effectivement la quantité de données utile à modèle constant.

Vocabulaire des coups. Nous énumérons géométriquement l'ensemble des coups concevables au format UCI (déplacements de type dame, de type cavalier, et promotions), soit exactement 1968 coups. Chaque coup possède un index fixe : la prédiction du modèle est donc un problème de classification à 1968 classes.

Vérification de réversibilité. Un bug d'encodage ne fait pas planter le programme : le modèle apprend quand même, mais mal, sans que rien ne le signale. Nous avons donc un test automatique qui reconstruit la position à partir des 68 nombres et vérifie qu'elle est identique à l'originale, ainsi qu'un test qui vérifie que le coup-cible est toujours légal dans sa position source.

4.3 Le modèle

Un Transformer encodeur (de type BERT, pas GPT : on classe, on ne génère pas de séquence).

HYPERPARAMÈTRE	VALEUR
Dimension interne (d_model)	256
Nombre de couches	8
Têtes d'attention	8
Dimension feed-forward	1024
Dropout	0,1
Nombre total de paramètres	6 858 416

Choix techniques notables :

- **Pré-normalisation** (LayerNorm avant l'attention) pour un entraînement plus stable sur un réseau profond.
- **Agrégation par moyenne** des 68 vecteurs de sortie avant la couche de classification.
- **Masquage des coups illégaux** : à l'inférence, tous les coups hors de la liste des coups légaux reçoivent un score de $-\infty$ avant le choix du maximum. Le bot ne peut donc structurellement pas jouer un coup illégal.

```
# src/engine/neural_bot.py (extrait) : masquage des coups illégaux
logits = self.model(tokens)                                # score pour les 1968
legal_mask = build_legal_mask(board, VOCAB)                # True pour les coup
logits = logits.masked_fill(~legal_mask, -float("inf"))
move = VOCAB.move_at(int(logits.argmax()))
```

À propos de flash-attn. La bibliothèque `flash-attn` figurait dans la stack imposée. Elle nécessite un GPU d'architecture Ampere ou plus récente et ne s'installe pas sur le GPU T4 gratuit dont nous disposons. Nous utilisons l'implémentation équivalente intégrée à PyTorch 2.x (`scaled_dot_product_attention`), qui sélectionne automatiquement le noyau d'attention optimisé disponible sur le matériel utilisé.

4.4 L'entraînement

- **Tâche** : classification multi-classes. Entrée = position, cible = index du coup joué. Fonction de perte : entropie croisée.
- **Optimiseur** : AdamW, taux d'apprentissage 3×10^{-4} , weight decay 0,01.
- **Planification du taux d'apprentissage** : montée linéaire (warmup) puis décroissance en cosinus.
- **Taille de lot** : 512.
- **Précision mixte** sur GPU pour accélérer le calcul.
- **Sauvegarde à chaque époque** sur Google Drive (résilience aux déconnexions de l'environnement Colab, voir §6).

4.5 L'application

Livrable central du projet : une application web sans dépendance externe (serveur Python standard, échiquier en CSS), avec deux modes :

- **Mode Jouer** : l'utilisateur affronte le Transformer ou l'un des bots de référence, coups légaux surlignés au clic.
- **Mode spectateur** : deux bots s'affrontent automatiquement, utile pour une démonstration sans manipulation.

Fonctionnalité clé : à chaque coup, la zone « coups envisagés » affiche les probabilités réellement calculées par le réseau pour ses meilleures options, donnant une vision directe de son raisonnement interne.

(Le code complet de l'application, du pipeline de données et de l'entraînement est disponible dans le dépôt GitHub, voir §10.)

5. Tests et résultats

5.1 Tests automatisés

Le projet comporte 32 tests unitaires (`pytest`), couvrant :

- la génération et la bijection du vocabulaire de 1968 coups ;
- la réversibilité de l'encodage et la légalité du coup-cible dans sa position source ;
- le parseur PGN (filtres, découpage entraînement/validation) ;
- le modèle (formes des tenseurs, sauvegarde/chargement).

Ces tests s'exécutent en moins de 30 secondes et sont relancés à chaque modification significative du code, ce qui a permis de détecter plusieurs régressions avant qu'elles n'affectent un entraînement de plusieurs heures.

5.2 Étalonnage des adversaires de référence

Avant d'évaluer le Transformer, nous validons notre chaîne de mesure sur trois bots simples. Le classement obtenu doit être cohérent avec leur niveau attendu.

MATCH (40 PARTIES)	BILAN	SCORE
Glouton vs Aléatoire	35 V / 5 N / 0 D	93,8 %
Minimax-2 vs Aléatoire	38 V / 2 N / 0 D	97,5 %
Minimax-2 vs Glouton	40 V / 0 N / 0 D	100 %

La hiérarchie **Minimax** > **Glouton** > **Aléatoire** est vérifiée : la chaîne d'évaluation est fiable.

5.3 Protocole de mesure de l'Elo

- Couleurs **strictement alternées** (l'avantage des blancs, ~55 %, biaiserait sinon la mesure).
- Ouvertures **variées** (sinon deux bots déterministes rejouent la même partie).
- **Intervalle de confiance de Wilson à 95 %** : à 100 % de victoires, l'intervalle classique donnerait un Elo infini ; Wilson fournit une borne finie et sensée.
- Sauvegarde de toutes les parties en PGN (preuve et analyse a posteriori).

5.4 Courbes d'entraînement

(Insérer ici la figure `courbes_entrainement.png` : perte, top-1, top-5.)

Résultats de l'entraînement (4 époques sur 1 million de positions, GPU T4) :

ÉPOQUE	PERTE (VAL)	TOP-1	TOP-5
1	3,95	12,6 %	35,4 %
2	3,39	18,4 %	45,4 %
3	3,14	21,9 %	50,5 %
4	3,08	22,7 %	51,7 %

La perte décroît régulièrement et les deux courbes (entraînement et validation) restent proches : le modèle n'est pas en surapprentissage. Chaque époque prend environ 8-9 minutes sur le GPU T4 gratuit.

Point d'interprétation important. Une top-1 de 20-40 % n'est pas un mauvais score. Dans beaucoup de positions, plusieurs coups sont de qualité équivalente : le modèle en propose un autre que l'humain sans avoir tort. La top-1 mesure l'imitation, pas la qualité de jeu. La seule mesure de force fiable est l'Elo constaté en parties réelles (§5.5).

Contrainte matérielle rencontrée. Les courbes étant encore croissantes à la 4^e époque, un entraînement plus long aurait vraisemblablement amélioré le modèle. Nous avons tenté un entraînement en 10 époques, mais l'environnement Colab gratuit se déconnecte au bout de 30 à 40 minutes, interrompant systématiquement une exécution de ~90 minutes. Pour garantir un résultat reproductible et livrable, nous avons retenu le modèle de 4 époques (top-1 22,7 %), sauvegardé sur Google Drive (voir §6).

5.5 Niveau du Transformer face à Stockfish

Campagne d'évaluation complète (60 parties par adversaire, couleurs alternées, intervalles de confiance de Wilson à 95 %) :

ADVERSAIRE	ELO ADV.	V	N	D	SCORE	ELO ESTIMÉ (IC 95 %)
Aléatoire	250	3	51	6	47,5 %	233 [146 ; 320]
Glouton	600	0	11	49	9,2 %	202 [54 ; 349]
Minimax profondeur 2	1100	0	2	58	1,7 %	392 [88 ; 695]
Minimax profondeur 3	1300	0	2	58	1,7 %	592 [288 ; 895]
Stockfish bridé 1320	1320	0	4	56	3,3 %	735 [507 ; 963]

Lecture des résultats. La mesure la plus directe est celle face à l'adversaire aléatoire : le bot y obtient 47,5 %, soit un niveau statistiquement équivalent au hasard (environ 230 Elo). Contre tous les adversaires structurés, il perd la quasi-totalité de ses parties. Face à Stockfish bridé à 1320, le niveau le plus faible que le moteur accepte, il ne gagne aucune partie mais en sauve quatre par la nulle, ce qui place son niveau nettement en dessous de 1320.

Un trait de comportement domine tous les matches : le très grand nombre de nulles (51 sur 60 contre l'aléatoire). Le bot atteint souvent des positions gagnantes sans les convertir : il déplace ses pièces sans plan, et les parties s'achèvent par répétition, règle des 50 coups, ou limite de coups. C'est la signature d'un jeu passif, cohérente avec une exactitude d'imitation modeste et un entraînement volontairement court.

Sur la dispersion des estimations. Les Elo estimés varient de 202 à 735 selon l'adversaire. Cet écart ne traduit pas une incohérence de la mesure, mais l'imprécision des Elo supposés des baselines, qui sont des ordres de grandeur admis et non des valeurs calibrées officiellement. La mesure directe face à l'aléatoire reste la plus fiable et situe le bot autour de 230-300 Elo, soit un niveau de tout premier débutant.

(Résultats reproductibles : `python -m scripts.evaluate --games 60` . Les parties sont sauvegardées en PGN dans `results/games/` , le rapport détaillé dans `results/elo_report.md` , fourni en annexe.)

5.6 Le modèle « comprend »-il les échecs ?

Une manière parlante de le vérifier au-delà des chiffres : demander au réseau ce qu'il propose dans la position de départ.

COUP	PROBABILITÉ
d3	31,9 %
Cc3 (Nc3)	16,8 %
Cf3 (Nf3)	14,3 %
d4	9,5 %
e3	8,7 %
c4	7,6 %

C'est un résultat marquant. Le modèle n'a jamais reçu la moindre règle du jeu : il n'a fait qu'observer des parties. Pourtant, il concentre ses propositions sur des coups d'ouverture parfaitement sensés : développement des cavaliers (Cf3, Cc3), occupation du centre (d4, c4). Il ne propose pas de coups absurdes comme a3 ou h4. Autrement dit, le réseau a appris implicitement des principes d'ouverture par simple imitation. Cette capacité contraste avec sa faiblesse en jeu réel : il sait commencer une partie, mais ne sait pas la conduire jusqu'à la victoire, ce qui répond directement à notre problématique de départ (§1.2) : oui, un Transformer sans recherche apprend quelque chose de réel, mais cet apprentissage a des limites nettes que la seule imitation ne comble pas.

(Cette sortie est directement visible dans l'application, zone « coups envisagés par le bot ».)

6. Difficultés rencontrées et solutions apportées

Débit de lecture des données trop lent. Notre première implémentation du parseur PGN traitait environ 400 parties/seconde, ce qui aurait demandé plusieurs heures pour extraire un million de positions. En analysant le problème, nous avons identifié que la bibliothèque standard construisait un arbre de variantes complet pour chaque partie, y compris celles rejetées ensuite par nos filtres. Solution : un lecteur qui inspecte d'abord les en-têtes (Elo, cadence) et ne parse les coups que pour les parties retenues, ce qui a multiplié le débit par ~18 (7 500 parties/seconde).

Elo infini à 100 % de victoires. Le calcul d'Elo classique (formule de Wald) devient mathématiquement indéfini quand un bot gagne toutes ses parties (division par zéro). Solution : remplacement par l'intervalle de confiance de Wilson, qui reste borné même dans les cas extrêmes et fournit une estimation exploitable (une borne, à défaut d'une valeur exacte).

Déconnexions de l'environnement d'entraînement gratuit. Google Colab (GPU gratuit) interrompt les sessions inactives ou trop longues après 30 à 40 minutes, ce qui a fait échouer plusieurs tentatives d'entraînement plus long que prévu, avec perte totale du travail en cours. Solution : sauvegarde automatique des poids du modèle sur Google Drive à la fin de chaque époque, permettant de reprendre sans tout recommencer, et choix final d'un nombre d'époques compatible avec la durée d'une session.

Fuite de données potentielle entre entraînement et validation. En concevant le découpage des données, nous avons identifié que répartir des positions individuelles au hasard entre les deux jeux aurait laissé des positions très proches (issues de la même partie) des deux côtés, faussant artificiellement les métriques de validation. Solution : découpage par partie entière, jamais par position isolée (voir §4.1).

[À compléter : ajoutez ici les difficultés que vous avez vécues personnellement pendant le développement, l'entraînement ou la rédaction, et comment vous les avez résolues. Les quatre points ci-dessus sont réels mais génériques au binôme ; le jury attend aussi des difficultés propres à chacune de vous.]

7. Bilan personnel et compétences acquises

Cette section doit être écrite par vous, avec vos propres mots. Le jury attend une réflexion personnelle authentique, pas une description technique de plus. Le squelette ci-dessous propose une structure ; les réponses doivent venir de vous.

7.1 Bilan de SEIBOU Naadjath

[Brouillon à relire et corriger avant dépôt : ce texte part de ce que j'ai réellement vécu pendant le projet, mais il doit passer par tes mots avant d'être rendu. Coupe, corrige, ajoute ce qui manque.]

Ce que ce projet m'a apporté. Je suis partie de très loin sur les deux sujets du projet à la fois : je n'avais jamais écrit de programme Python complet, et je ne savais pas jouer aux échecs. Construire un bot d'échecs en partant de ces deux manques m'a obligée à apprendre les deux en parallèle, souvent dans l'urgence, plutôt que dans l'ordre confortable « d'abord la théorie, ensuite la pratique ». Le changement le plus net dans ma façon de voir les choses concerne l'utilisation de l'IA comme outil de travail : je me suis assurée de comprendre et de pouvoir réexpliquer chaque partie du code plutôt que de la laisser telle quelle, précisément parce que je savais que je devrais la défendre à l'oral. C'est une discipline différente de « faire fonctionner » un projet.

Compétences techniques mobilisées ou développées. Prise en main de Python en partant de zéro, notamment la manipulation de fichiers, les tests automatisés (pytest) et l'organisation d'un projet en modules. Découverte concrète du fonctionnement d'un réseau de neurones de type Transformer, au-delà de la théorie : voir une courbe de perte descendre réellement, comprendre pourquoi une exactitude de 22 % n'est pas un mauvais score. Prise en main de Git/GitHub pour un travail à deux avec un historique de commits partagé.

Compétences professionnelles/transversales. Gestion d'un projet technique long soumis à des imprévus : les déconnexions répétées de l'environnement d'entraînement gratuit (Google Colab) ont demandé de revoir la stratégie plusieurs fois plutôt que d'abandonner, et une confusion sur la date réelle de rendu a demandé de corriger rapidement plusieurs documents déjà avancés. Ces deux épisodes m'ont appris à distinguer un blocage réel d'un contretemps à contourner.

Points que je souhaiterais approfondir. Mieux comprendre les Transformers de façon plus générale, au-delà de leur application aux échecs : leur usage en traitement du langage et dans d'autres domaines.

7.2 Bilan de LAKRA Rajaa

[ATTENTION, BROUILLON NON VÉRIFIÉ. Je n'ai aucune information directe sur ce que Rajaa a vécu pendant ce projet : ce texte propose des formulations plausibles pour quelqu'un dans sa situation, pas des faits. Rajaa doit le lire et corriger tout ce qui ne lui correspond pas avant le dépôt, y compris si ça veut dire tout réécrire. Ne pas rendre tel quel.]

Ce que ce projet m'a apporté. Comme Naadjath, je n'avais pas une pratique approfondie de Python avant ce projet, et je ne connaissais pas les règles précises du jeu d'échecs au-delà des bases. Travailler sur un sujet technique aussi exigeant m'a obligée à apprendre en marchant plutôt qu'en suivant un cours structuré, en m'appuyant sur ce que Naadjath comprenait déjà et en cherchant par moi-même sur les points que je ne maîtrisais pas.

Compétences techniques mobilisées ou développées. Manipulation de Python sur un projet réel plutôt que sur des exercices isolés, découverte du fonctionnement d'un moteur d'échecs externe (Stockfish) et de son intégration via le protocole UCI, prise en main de Git pour un travail en binôme avec un historique de commits partagé.

Compétences professionnelles/transversales. Travail en équipe sur un projet long avec des imprévus techniques (notamment les déconnexions de l'environnement d'entraînement gratuit), qui a demandé de s'adapter et de prioriser plutôt que de suivre un plan figé. Communication avec ma binôme pour synchroniser l'avancement sur les différentes parties du projet.

Points que je souhaiterais approfondir. [à compléter par Rajaa : par exemple l'évaluation de modèles, le développement d'applications, ou tout autre aspect du projet qui l'a particulièrement intéressée.]

8. Conclusion et perspectives

8.1 Conclusion

Nous avons montré qu'un Transformer de taille modeste peut apprendre à jouer aux échecs sans aucune recherche, à partir de la seule observation de parties. Le niveau atteint reste modeste, de l'ordre de 230 à 300 Elo, soit un tout premier niveau de débutant, mais le modèle acquiert des principes d'ouverture réels, et la démarche est complète et rigoureuse : chaîne de données reproductible, encodage vérifié par des tests automatisés, entraînement suivi, et surtout évaluation honnête avec intervalles de confiance plutôt que des chiffres présentés sans nuance.

Les objectifs fixés en introduction sont atteints : un Transformer fonctionnel a été conçu et entraîné, son niveau a été mesuré rigoureusement face à Stockfish, une application permet de le tester en direct, et l'ensemble est documenté et reproductible. Le niveau de jeu, en revanche, reste en deçà de ce qu'on pourrait espérer d'un « bot d'échecs » au sens courant : c'est un résultat que nous assumons et analysons plutôt que de le dissimuler.

Le principal enseignement dépasse le score obtenu : il illustre la différence entre **connaissance** (ce que le réseau a mémorisé en observant des parties) et **calcul** (la recherche que nous avons volontairement supprimée), et montre concrètement les limites de la première sans la seconde.

8.2 Perspectives

- **Passer à l'action-value** (prédire la probabilité de gain de chaque coup) comme DeepMind, plutôt que d'imiter le coup joué par l'humain.

- **Annoter les données avec Stockfish** pour apprendre du meilleur coup possible plutôt que du coup effectivement joué, qui peut être imparfait.
 - Ajouter une **recherche légère** (alpha-bêta de profondeur 2-3) guidée par les coups que propose le réseau, pour corriger l'essentiel des erreurs tactiques observées.
 - **Entraîner plus longtemps** sur un environnement GPU stable (non sujet aux déconnexions), les courbes de la §5.4 étant encore croissantes en fin d'entraînement.
 - Faire jouer le bot sur Lichess via l'API bot pour obtenir un Elo officiel, calibré sur de vrais joueurs plutôt que sur des baselines approximatives.
-
-

9. Bibliographie et sources

1. A. Ruoss, G. Delétang, S. Medapati, J. Grau-Moya, L. K. Wenliang, E. Catt, J. Reid, T. Genewein, *Grandmaster-Level Chess Without Search*, DeepMind, 2024.
2. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, *Attention Is All You Need*, NeurIPS, 2017.
3. D. Silver, T. Hubert, J. Schrittwieser et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm* (AlphaZero), 2017.
4. Documentation python-chess : <https://python-chess.readthedocs.io>
5. Base de données Lichess (source des parties d'entraînement) : <https://database.lichess.org>

6. Documentation Stockfish (moteur de référence pour l'évaluation) :
<https://stockfishchess.org>
7. Documentation PyTorch (bibliothèque d'apprentissage automatique utilisée pour le Transformer) : <https://pytorch.org/docs>

(Les jeux de données, bibliothèques et API listés ci-dessus sont les seules sources externes utilisées ; le code d'implémentation est un travail original du binôme, documenté et versionné sur GitHub, voir §10.)

10. Annexes et GitHub

10.1 Dépôt GitHub

Code source complet, historique des commits et documentation technique :

<https://github.com/naadjath/chess-bot-v1>

Le dépôt contient un fichier `README.md` qui explique le projet, les technologies utilisées, l'architecture du code et les modalités d'exécution et de test de la solution (installation, lancement de l'application, relance de l'entraînement, relance de la campagne d'évaluation).

10.2 Documents en annexe

- `results/elo_report.md` : rapport détaillé de la campagne d'évaluation Elo (données brutes derrière le tableau du §5.5).
- `results/games/*.pgn` : l'intégralité des parties jouées pendant la campagne d'évaluation, au format standard PGN, consultables dans n'importe quel logiciel d'échecs.
- `checkpoints/best.pt` : les poids entraînés du modèle.
- `checkpoints/history.json` : les métriques d'entraînement complètes (données derrière les courbes du §5.4).

10.3 Reproduire nos résultats

```
git clone https://github.com/naadjath/chess-bot-v1.git
cd chess-bot-v1
pip install -r requirements.txt

python -m pytest tests/ -v                # 32 tests
python -m scripts.prepare_data --demo-games 60    # pipeline de donn
python -m scripts.evaluate --games 60            # campagne d'evalu
python -m src.app.server                      # lancer l'applica
```